Experiment 08 Report

R.VIKAS RAJ
EE19B108
BATCH 1

Aim :

1. To understand C-interfacing (use C-programming) in an ARM platform.

2. To study and implement ADC / DAC in the ARM platform.

R:equirements :

1. LPC2148 ARM Development board and accessories.

2. RS-232 cable

3. Keil microvision 5 (C - interface)

4. flash magic

5. Burn o-mat

6. DSO (Digital Storage Oscilloscope)

7. Sample programs for generating digital inputs. (For analog inputs, potentiometer is used)

Tasks to do :

1 Analog-to-Digital Conversion (ADC):
Take a real-time analog signal from a sensor and convert it into a digital signal by implementing an ADC. Reduce the step size and observe any changes in the number of bits needed to represent the entire range. Determine the quantization error.

2 Digital-to-Analog Conversion (DAC):
Using the LPC2148 ARM development board and a provided square wave program, modify it to generate the following waveforms:

1. Ramp (sawtooth) wave
2. Triangular wave

3. Sine wave (either through a lookup table or by using formulas)

Task 1 - Analog-to-Digital Conversion (ADC):

Code:

```
#include <LPC214x.H> /* LPC214x definitions */ #include
"delay.h"
unsigned long Read_ADC0(unsigned char); /*
void Init_ADC0(unsigned char);
#ifndef __ADC_H
#define __ADC_H
#define CHANNEL_0 0
#define CHANNEL_1 1
#define CHANNEL_2 2
#define CHANNEL_3 3
#define CHANNEL_4 4
#define CHANNEL_5 5
#define CHANNEL_6 6
#define CHANNEL_7 7
/* Crystal frequency,10MHz~25MHz should be the same as actual status. */
#define Fosc 12000000 /* 12 MHz is the operational frequency of o/p dgtlClk
*/
#define ADC_CLK 1000000 /* set to 1Mhz */ /* A/D Converter
0 (AD0) */
#define AD0_BASE_ADDR 0xE0034000
#define ADC_INDEX 4
#define ADC_DONE 0x80000000
#define ADC_OVERRUN 0x40000000
```

```
#define ADC_FullScale_Volt 3.3 // 3.3V - ADC Referance Voltage
#define ADC_FullScale_Count 1024 // 2^10 - 10 bit ADC
#define LED_IOPIN IO0PIN
#define BIT(x) (1 << x)
#define LED_D0 (1 << 10) // P0.10 mapping same as in Exp7 switch LED #define
LED_D1 (1 << 11) // P0.11
#define LED_D2 (1 << 12) // P0.12
#define LED_D3 (1 << 13) // P0.13
#define LED_D4 (1 << 15) // P0.15
#define LED_D5 (1 << 16) // P0.16
#define LED_D6 (1 << 17) // P0.17
#define LED_D7 (1 << 18) // P0.18
#define LED_DATA_MASK ((unsigned long)((LED_D7 | LED_D6 | LED_D5 | LED_D4 |
 LED_D3 | LED_D2 | LED_D1 | LED_D0)))
#define LED1_ON LED_IOPIN |= (unsigned long)(LED_D0); // LED1 ON
#define LED2_ON LED_IOPIN |= (unsigned long)(LED_D1); // LED2 ON
#define LED3_ON LED_IOPIN |= (unsigned long)(LED_D2); // LED3 ON
#define LED4_ON LED_IOPIN |= (unsigned long)(LED_D3); // LED4 ON
#define LED5_ON LED_IOPIN |= (unsigned long)(LED_D4); // LED5 ON
#define LED6_ON LED_IOPIN |= (unsigned long)(LED_D5); // LED6 ON
#define LED7_ON LED_IOPIN |= (unsigned long)(LED_D6); // LED7 ON
#define LED8_ON LED_IOPIN |= (unsigned long)(LED_D7); // LED8 ON
#endif
        #ifndef LED_DRIVER_OUTPUT_EN
#define LED_DRIVER_OUTPUT_EN (1 << 5) // P0.5
#endif


}
//LED definitions
int main (void)
```

```
{
        unsigned long ADC_val;
        Init_ADC0(CHANNEL_1);
        Init_ADC0(CHANNEL_2);
        delay_mSec(100);
        IO0DIR |= LED_DATA_MASK; // GPIO Direction control -> pin is output
        IO0DIR |= LED_DRIVER_OUTPUT_EN; // GPIO Direction control -> pin is
output
        IO0CLR |= LED_DRIVER_OUTPUT_EN;
        while(1)
        {
                //ADC_val = Read_ADC0(CHANNEL_1);
                ADC_val = Read_ADC0(CHANNEL_2);
                ADC_val=(ADC_val>>2);
                delay_mSec(5);
        if(ADC_val & BIT(0)) LED8_ON;
        if(ADC_val & BIT(1)) LED7_ON;
        if(ADC_val & BIT(2)) LED6_ON;
        if(ADC_val & BIT(3)) LED5_ON;
        if(ADC_val & BIT(4)) LED4_ON;
        if(ADC_val & BIT(5)) LED3_ON;
        if(ADC_val & BIT(6)) LED2_ON;
        if(ADC_val & BIT(7)) LED1_ON;
        }
// return 0;
}
void Init_ADC0(unsigned char channelNum)
```



```
{
```

```
        if(channelNum == CHANNEL_1)
                        PINSEL1 = (PINSEL1 & ~(3 << 24)) | (1 << 24); // P0.28 -> AD0.1

        if(channelNum == CHANNEL_2)
                PINSEL1 = (PINSEL1 & ~(3 << 26)) | (1 << 26); // P0.29 -> AD0.2
        if(channelNum == CHANNEL_3)
                        PINSEL1 = (PINSEL1 & ~(3 << 28)) | (1 << 28); // P0.30 -> AD0.3
   AD0CR = ( 0x01 << 1 ) | // SEL=1, select channel 0, 1 to 4 on ADC0
                          (( Fosc / ADC_CLK - 1 ) << 8 ) | // CLKDIV = Fpclk / 1000000 - 1
                                                  ( 0 << 16 ) | // BURST = 0, no
BURST, software controlled
                                                  PDN = 1, normal
clocks/10 bits operation
                                                  ( 0 << 22 ) | // TEST1:0 = 00 ( 0 << 24 ) | //
                                                  START = 0 A/D
conversion stops
( 0 << 17 ) | // CLKS = 0, 11 ( 1 << 21 ) | //  ( 0 << 27 ); /* EDGE = 0

(CAP/MAT singal falling,trigger A/D conversion) */
}


unsigned long Read_ADC0( unsigned char channelNum )
{
    unsigned long regVal, ADC_Data;
        /* Clear all SEL bits */
   AD0CR &= 0xFFFFFF00;
        /* switch channel, start A/D convert */
   AD0CR |= (1 << 24) | (1 << channelNum);
   /* wait until end of A/D convert */
        while ( 1 ) {
```

```
// regVal = *(volatile unsigned long *)(AD0_BASE_ADDR + ADC_INDEX); regVal =
                    AD0GDR;
                    if ( regVal & ADC_DONE ){
                    break;
                    }
   }
        /* stop ADC now */
   AD0CR &= 0xF8FFFFFF;
        /* save data when it's not overru otherwise, return zero */
    if ( regVal & ADC_OVERRUN ) {
                    return ( 0 );
   }
   ADC_Data = ( regVal >> 6 ) & 0x3FF;
   /* return A/D conversion value */
        return ( ADC_Data );
}
void delay_mSec(int dCnt) // pr_note:~dCnt mSec
{
  int j=0,i=0;
  while(dCnt--)
  {
           for(j=0;j<1000;j++)
           {
             /* At 60Mhz, the below loop introduces
             delay of 10 us */
             for(i=0;i<10;i++);
           }
  }
}
```

Explanation:

To convert a real-time analog signal from a sensor into a digital signal, we implement an Analog-to-Digital Converter (ADC). The ADC takes continuous analog values and quantizes them into discrete digital levels. This involves sampling the analog signal at a specified frequency and converting each sample to a binary value that represents the signal's amplitude at that moment.

Decreasing the step size:

When we decrease the step size, which refers to the smallest difference between two quantization levels, the ADC becomes more precise. A smaller step size requires a higher resolution, which means more bits are needed to represent the same range of values. This increased bit depth allows the ADC to capture finer variations in the analog signal.

Quantization error:

As step size decreases, quantization error, which is the difference between actual analog value and the quantized digital value, also decreases. Quantization error is inherently present in ADC processes due to the rounding of continuous values to discrete levels, but by increasing resolution (decreasing step size), this error is minimized, resulting in a more accurate digital representation of analog signal.

Task 2 - Digital-to-Analog Conversion (DAC):

1) **Ramp (sawtooth) wave:**

Code:

```
// DAC program for LPC2148 SUNTECH KIT


#include "LPC214x.H" /* LPC214x definitions */


#define DAC_BIAS 0x00010000
```

```
void mydelay(int);



7
void DACInit(void) {
   /* setup the related pin to DAC output */
   PINSEL1 &= 0xFFF3FFFF;
   PINSEL1 |= 0x00080000; /* set p0.25 to DAC output */
   return;
}

int main(void) {
   DACInit();
   unsigned int i;

   while (1) {
      // Ramp up
      for (i = 0; i <= 0x3FF; i++) {
         DACR = (i << 6) | DAC_BIAS;
         mydelay(0x01); // Adjust delay for smoothness
      }

      // Ramp down
      for (i = 0x3FF; i > 0; i--) {
         DACR = (i << 6) | DAC_BIAS;
         mydelay(0x01); // Adjust delay for smoothness
      }
   }
}
```

Explanation:

The square wave generator code can be modified a little in the while loop to give rise to the ramp (sawtooth) wave.

8

DAC_Init:

This function sets p0.25 to DAC output.

Inside the while loop:

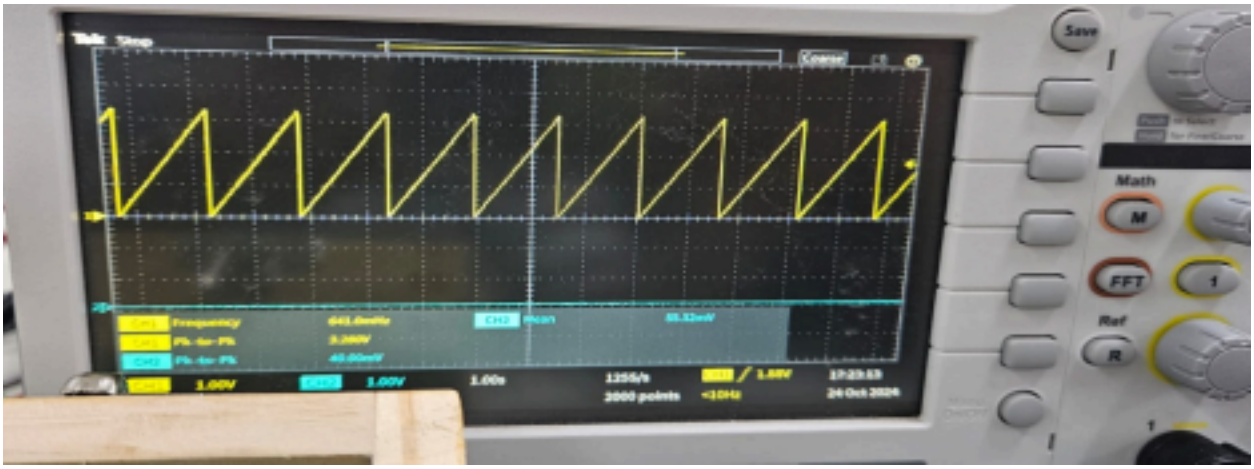
The `for` loop starts with `i = 0` and increments `i` up to `0x3FF` (1023 in decimal).

In each iteration, the DAC output register (DACR) is set to `i << 6 | DAC_BIAS`. The `i << 6` operation shifts `i` left by 6 bits, setting the DAC's output voltage based on `i`'s value.

The `mydelay(0x01);` function introduces a small delay, controlling the speed of the ramp.

The outer `while (1)` loop causes the ramp-up to repeat indefinitely, creating a continuous ramp wave output.

Output seen through the oscilloscope:



2) Triangular wave:

Code:

```
#include "LPC214x.H" /* LPC214x definitions */


g
#define DAC_BIAS 0x00010000
void mydelay(int);

void DACInit(void) {
   /* setup the related pin to DAC output */
   PINSEL1 &= 0xFFF3FFFF;
   PINSEL1 |= 0x00080000; /* set p0.25 to DAC output
   */ return;
}

int main(void) {
   DACInit();
   unsigned int i;

   while (1) {
      // Ramp up
      for (i = 0; i <= 0x3FF; i++) {
         DACR = (i << 6) | DAC_BIAS;
         mydelay(0x01); // Adjust delay for frequency
      control }

      // Ramp down
```

```
        for (i = 0x3FF; i > 0; i--) {
            DACR = (i << 6) | DAC_BIAS;
            mydelay(0x01); // Adjust delay for frequency
        control }
    }
}
```

10

```
void mydelay(int x) {
    int j, k;
    for (j = 0; j <= x; j++) {
        for (k = 0; k <= 0xFF; k++);
    }
}
```

Explanation:

The square wave generator code can be modified a little in the while loop to give rise to the triangular wave.

DAC_Init:

This function sets p0.25 to DAC output.

Inside the while loop:

1. Ramp Up Phase:
    - The first `for` loop starts with `i = 0` and increments `i` up to `0x3FF` (1023 in decimal).
    - In each iteration, the DAC output register (DACR) is set to `i << 6 | DAC_BIAS`. The `i << 6` operation shifts `i` left by 6 bits, setting the DAC's output voltage based on `i`'s value.
    - The `mydelay(0x01);` function introduces a small delay, controlling the speed of the ramp.
2. Ramp Down Phase:

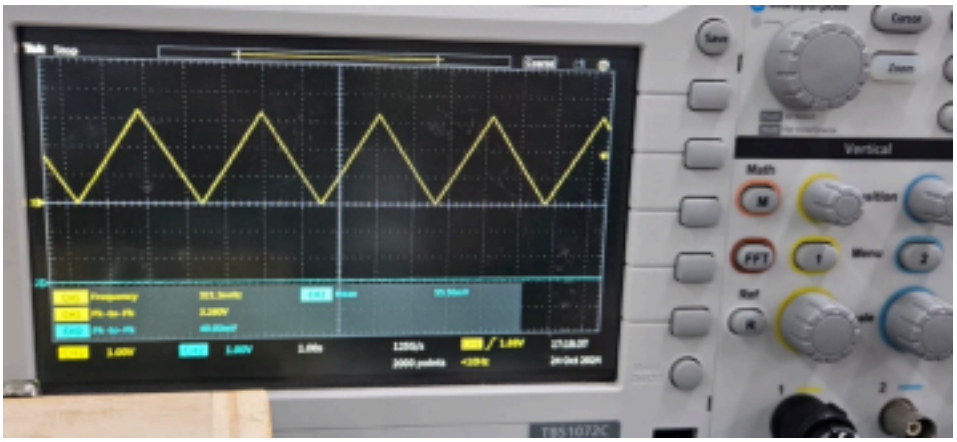
- The second `for` loop starts with `i = 0x3FF` and decrements `i` down to `0`.
- Similar to the ramp-up phase, `DACR` is set to `i << 6 | DAC_BIAS`, and `mydelay(0x01);` provides a delay between steps.

3. Loop Continuation:
   - The outer `while (1)` loop causes the ramp-up and ramp-down phases to repeat indefinitely, creating a continuous ramp wave output.

The output is a triangular waveform where the DAC output voltage ramps up linearly and then ramps down linearly, controlled by the increment and decrement of `i`.

Output seen through the oscilloscope:

ll



3) Sine wave:

Code:

```
// DAC program for LPC2148 SUNTECH KIT


#include "LPC214x.H" /* LPC214x definitions */


#define DAC_BIAS 0x00010000
void mydelay(int);


void DACInit(void) {
   /* setup the related pin to DAC output */
```

```
    PINSEL1 &= 0xFFF3FFFF;
    PINSEL1 |= 0x00080000; /* set p0.25 to DAC output */
    return;
}

#define SINE_TABLE_SIZE 256

unsigned int sine_table[SINE_TABLE_SIZE] = {
    127,130,133,136,139,143,146,149,152,155,158,161,164,167,170,173,176

 , 12

    178,181,184,187,190,192,195,198,200,203,205,208,210,212,215,217,219
    ,
    221,223,225,227,229,231,233,234,236,238,239,240,242,243,244,245,247,
    248,249,249,250,251,252,252,253,253,253,254,254,254,254,254,254,254,
    253,253,253,252,252,251,250,249,249,248,247,245,244,243,242,240,239,
    238,236,234,233,231,229,227,225,223,221,219,217,215,212,210,208,205,
    203,200,198,195,192,190,187,184,181,178,176,173,170,167,164,161,158,
    155,152,149,146,143,139,136,133,130,127,124,121,118,115,111,108,105,
    102,99,96,93,90,87,84,81,78,76,73,70,67,64,62,59,56,54,51,49,46,44,
    42,39,37,35,33,31,29,27,25,23,21,20,18,16,15,14,12,11,10,9,7,6,5,5,
    4,3,2,2,1,1,1,0,0,0,0,0,0,0,1,1,1,2,2,3,4,5,5,6,7,9,10,11,12,14,15,
    16,18,20,21,23,25,27,29,31,33,35,37,39,42,44,46,49,51,54,56,59,62,
    64,67,70,73,76,78,81,84,87,90,93,96,99,102,105,108,111,115,118,121,
    124
};
int main(void) {
    DACInit();
```

```
    unsigned int i = 0;


    while (1) {
        DACR = (sine_table[i] << 6) | DAC_BIAS; // Output sine wave value to DAC
        mydelay(0x02); // Adjust delay to control frequency
        i++;
        if (i >= SINE_TABLE_SIZE) {
            i = 0; // Wrap around the table
        }
    }
}
void mydelay(int x) {
    int j, k;


13
    for (j = 0; j <= x; j++) {
        for (k = 0; k <= 0xFF; k++);
    }
}
```

Explanation:

The square wave generator code can be modified a little in the while loop to give rise to the sine wave.

DAC_Init:

This function sets p0.25 to DAC output.

Inside the while loop:

Outputting Sine Values:

- The code reads values from `sine_table` at index `i` and shifts each value left by 6 bits (`sine_table[i] << 6`). This sets the appropriate DAC output level corresponding to the sine wave amplitude.
- The `DACR` register is set with this shifted value combined with

`DAC_BIAS` to account for any necessary offset or bias.

Delay Control:

- The `mydelay(0x02);` function introduces a delay between each DAC update. Adjusting this delay controls the frequency of the generated sine wave: a shorter delay results in a higher frequency, and a longer delay results in a lower frequency.

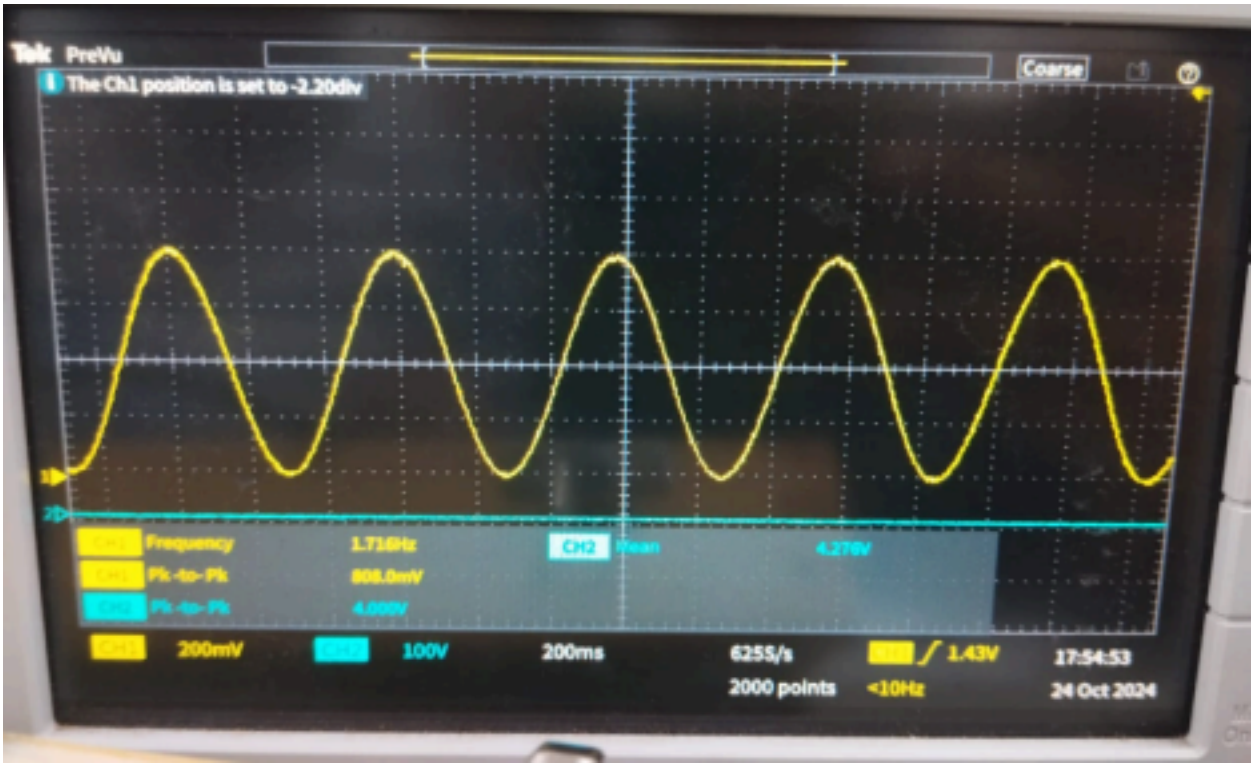
Index Increment and Wrapping:

- After each output, the code increments `i` to move to the next entry in the sine table.
- If `i` reaches the end of the table (`SINE_TABLE_SIZE`), it resets to `0`, effectively wrapping around to start the sine wave pattern from the beginning. This wrapping ensures a continuous loop through the sine values, producing a periodic sine wave output.

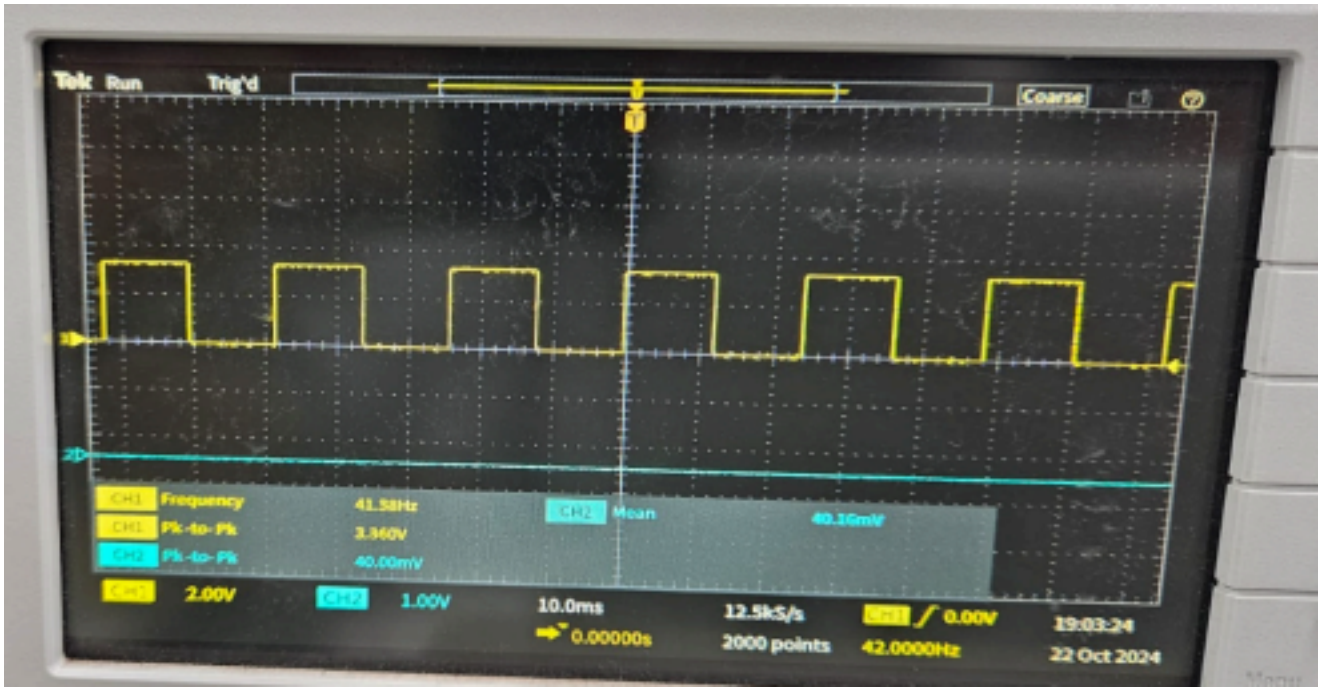
Infinite Loop:

- The `while (1)` loop causes this process to repeat indefinitely, allowing the DAC to output a continuous sine wave signal.

Output seen through the oscilloscope:



Square wave output seen through the oscilloscope:



Inferences and conclusions drawn from the experiment:

1. **Understanding ADC/DAC Operations**:

The experiment demonstrated how to implement Analog-to-Digital

Converters (ADC) and Digital-to-Analog Converters (DAC) on the ARM LPC2148 microcontroller. Students learned the fundamental concepts of converting analog signals to digital format and vice versa.

2. **Practical Skills in Microcontroller Programming**:

   By writing and modifying C programs to generate various waveforms (ramp, triangular, sine), students practiced programming in an embedded environment, specifically using the Keil µVision software and interfacing tools like Flash Magic for program upload.

3. **Generating and Observing Waveforms**:

   The experiment involved generating different waveforms (such as ramp, triangular, and sine waves) using DAC. This process helped students understand waveform generation and the effect of changing parameters (like delay) on waveform frequency.

4. **Quantization and Sampling Concepts**:

   The ADC section allowed students to observe the effects of quantization, sampling, and step size adjustments, gaining insights into resolution and quantization error in digital representations of analog signals.

5. **Hands-On Exposure with Hardware Setup**:

   The experiment provided hands-on experience with connecting and configuring the LPC2148 development board, using an oscilloscope to view output signals, and understanding the practical aspects of hardware interfacing.

This experiment helped me to gain both theoretical and practical knowledge in ADC/DAC functionality, waveform generation, and embedded system programming on ARM microcontrollers.